

What Your Pipeline Reports Is Not What It Ran: Silent Failure in an ARC-AGI-2 Solver

ARC Prize 2026 — Paper Track Writeup

Abstract

Pipelines report the work they intend to do, not the work they did. We instrumented a test-time-training (TTT) ARC-AGI-2 solver and found that its adaptation ran at **1.34 optimizer steps per task** against the reference recipe's 128 — **and that the run's own counter misreported** even that, recording **21** steps where the kernel log recorded **121**, disagreeing on **77 of 102** tasks. **Neither the accuracy nor the telemetry told the truth about how much adaptation happened.** We place this beside two sibling failures, and recommend the fix we first had to apply to ourselves: **count executed optimizer steps, and verify that counter against an independent channel.**

1. Introduction

ARC-AGI-2 asks for a rule inferred from a few demonstration pairs. Because each task *is* a tiny training set, test-time training is the dominant approach [1]. Our system follows that lineage — a 4B open model, per-task LoRA, thresholded beam search — the configuration NVARC won 2025 with [6]. The premise is that the model is adapted before it predicts. This paper is about the gap between that premise and what a run does.

Accuracy cannot close that gap: it is the product of *generating* the right answer and *selecting* it, and when it is low the natural inference is that the model is not good enough. We made that inference, then measured — and found adaptation running at about one percent of its budget, while neither the accuracy nor our own instrument said so.

2. Prior work

ARC-AGI-2 is harder and less brute-forcible than ARC-AGI-1 [8, 2]; the 2025 top score was 24.03% [9]. TTT-based transduction dominates — all top LLM-based entries use it [1] — and NVARC's winner is Qwen3-4B with LoRA [6], so our system is a scaled-down instance of an established recipe, not a new solver. Memory-bound TTT-for-ARC is documented: 16 GB P100 [3], L4×4 [10], rank-32 LoRA at batch 1 [5]. **We found no prior report of an adaptation stage whose execution was misreported by its own instrumentation, which is the claim we make.**

The generation/selection decomposition is not ours: ARChitects' "coverage" is "an upper bound for the performance of the selection algorithms" [5, Fig. 4]; Li et al. publish "Sample+Oracle" [4, Fig. 8]; Moghe & Chin make the split their headline [7]. Split-distribution mismatch is published too — 6.9% of train versus 40.8% of eval tasks need multiple answers [11], against our 7.1% and 40.8%.

3. Approach

System. Qwen3-4B fine-tuned to a 16-token grid vocabulary (3.634B parameters); per-task LoRA (rank 256, rsLoRA, all projections plus `embed_tokens`` and `lm_head``); cosine schedule; constrained beam search; cascade scheduling (cheap sweep, TTT on the weakest tasks, symbolic backfill). Every emitted grid is validated.

Instrumentation. `ttt_steps`` is incremented *inside* the optimizer loop, so it counts steps that ran. Pool recall asks whether the ground-truth grid appears anywhere in a task's candidate pool — borrowed practice (§2), used only to locate the loss.

Two earlier bugs had the same shape, both favouring the hypothesis under test: a pool-recall denominator inflated by per-stage accumulation (16 where 11 inputs existed), and a missing field read as "0 in pool". A diagnostic whose failure mode is *confirms what you hoped* is worse than none.

4. Results

4.1 We are generation-bound, with zero selection headroom

Pool recall was zero in all four instrumented runs: **0 of 35 (configuration, input) measurements — that is 8 evaluation tasks and 11 unique test inputs re-measured under three configurations**, not 35 independent trials. A zero count over 11 inputs bounds the true rate at **$\leq 23.8\%$ (95%)**; we give the bound, not the flattering point estimate. Pools held **4–29** distinct candidates, so "the pool was too small" is ruled out.

4.2 Search width is not the lever

Three paired configurations — identical tasks, budget, augmentations, seed. Halving the token probability threshold ($0.2 \rightarrow 0.1$) and doubling beam breadth ($3 \rightarrow 4$) and node budget ($6,000 \rightarrow 12,000$) changed nothing: 0/8 solved, 0.0% recall, at 30% more time per task ($203 \rightarrow 263$ s).

4.3 The adaptation ran at one percent of its budget, and the counter lied about it

In the 240-task run, Stage B was entered by **102** tasks and consumed **7,266 seconds**. Counting optimizer steps from the kernel log — the only channel that records them per attempt:

```
| measured from | tasks that stepped | total optimizer steps |  
|---|---|---|  
| the run's own report (`report.json`) | 16 / 102 | 21 |  
| the kernel log (ground truth) | 93 / 102 | 121 |
```

The two disagree on **77 of 102 tasks**. The report was wrong because `record_task` runs once per *stage*, and a later stage re-records the same task with a fresh result whose step count is zero, erasing what adaptation had measured. We wrote the counter to detect unexecuted adaptation, published a number from it, and it was wrong by a factor of six; only the log caught it.

What the corrected numbers say is subtler than "it never ran". Adaptation **did** execute, on 93 of 102 tasks — at a mean of **1.34 optimizer steps** against the reference recipe's **128**, a shortfall of two orders of magnitude. Across every run we made (294 tasks, 112 attempts), 12% of attempts executed no step at all.

Two masking paths remain. Without a circuit breaker each task fails independently and the run continues; with one, a single allocation failure disables the mechanism run-wide — the *reasonable* engineering choice, observed firing in three of seven runs, which escalates one task's failure into a systematically unadapted submission. Adaptation is an optional refinement inside a pipeline that must emit a submission regardless, so a resource failure degrades gracefully into "unadapted model": **graceful degradation is what makes it silent**, and a counter that is itself unreliable is what makes it undetectable.

4.4 The accelerator was wrong, and nothing said so

Everything ran on one Tesla T4 (14.56 GiB); the competition offers ``NvidiaL4`` — four L4s, 88 GiB. Our own compatibility guard *declared the L4 unusable*, testing membership in ``torch.cuda.get_arch_list()``, which omits ``sm_89``; real work succeeds on an L4. The memory pressure

above was therefore self-inflicted. The lesson is not the memory but the reporting: a wrong configuration produced a run whose error list was empty and whose output read as a capability result. We did not notice for a week — the same silence this paper is about.

4.5 The local validation loop was invalid

The public test file is not a holdout: all 240 tasks are byte-identical to training tasks, answers included, with zero overlap with the evaluation split. It also under-represents multi-answer tasks (7.1% versus 40.8%), so a builder that is correct locally is malformed on the hidden set. The structural half reproduces [11]; we claim no priority.

5. Discussion

What is new. Pool recall is not ours; the generation-bound finding is a replication, search width confirmatory, distribution mismatch a reproduction. **The misreported adaptation stage is the one claim we make.**

The safeguard. Count executed optimizer steps per task, and **verify that counter against a channel the pipeline does not control — the kernel log is what caught ours. Make degradation per-task rather than global, and never report a stage's null result without confirming the stage executed and that your measurement of it agrees with an independent source.** We learned both by violating them. The counter and its self-test ship as a dependency-free module.

6. Conclusion

We found that the part of our solver we believed in most was running at one percent of its budget, and that the run's own instrument concealed it. The contribution is the failure mode and its safeguard: optional learned refinement stages fail silently, and the fix is a counter checked against a channel the pipeline does not control.

Appendix A — Artefact and submission linkage

Solver, diagnostic tooling, raw run reports and kernel logs are public under MIT-0. Every number here is re-derived by ``tools/evidence_ledger.py``, which prints each cited value beside its source file; 92 checks require no GPU; all pass. The linked Kaggle notebook writes ``/kaggle/working/submission.json`` with internet disabled. The 240-task run cost 37,963 s on one T4 at 14.12 GiB peak. Leaderboard score: **0.42%** — a real scored entry, which the rules require for eligibility and explicitly do not require to be high.

References

1. ARC Prize 2024 Technical Report. arXiv:2412.04604
2. Chollet. On the Measure of Intelligence. arXiv:1911.01547
3. Cole & Osman. arXiv:2506.14276
4. Li et al. Combining Induction and Transduction. arXiv:2411.02272
5. ARChitects. Product of Experts. arXiv:2505.07859
6. NVARC. github.com/1ytic/NVARC
7. Moghe & Chin. arXiv:2607.06764
8. ARC-AGI-2. arXiv:2505.11831
9. ARC Prize 2025 Technical Report. arXiv:2601.10904
10. ARC Prize 2025 archive. arcprize.org/competitions/2025/archive
11. Habr, split-distribution analysis. habr.com/ru/articles/1071730/